Python Engineering - Class 05 Reference Guide

Lists, Tuples and Trajectories

Reference guide to the syntax and tools used in this class. It explains what each element does, how it is written and when it is useful.

Quick map

Concept	Syntax	Idea
list	values = []	Ordered mutable collection.
append()	values.append(x)	Adds one element at the end of a list.
indexing	values[i]	Select one element by position; Python starts at index 0.
negative index	values[-1]	Select from the end; -1 is the last element.
slicing	values[a:b]	Select a subsequence; b is excluded.
tuple	state = (h,v,t)	Ordered container typically used here as a compact state snapshot.
mutability	list mutable / tuple container immutable	Lists can change in place; tuple item references cannot be reassigned.
aliasing	a = b	Two names may refer to the same mutable object.
generator expression	max(s[0] for s in trajectory)	Generate values lazily for functions such as max().
lambda	lambda s: s[0]	Small anonymous function; useful as key= in some operations.

1. list

Syntax: `values = []`

Ordered mutable collection.

```
trajectory = []
```

Control question: before running the example, predict the result type and shape.

2. append()

Syntax: `values.append(x)`

Adds one element at the end of a list.

```
trajectory.append((h, v, t))
```

Control question: before running the example, predict the result type and shape.

3. indexing

Syntax: `values[i]`

Select one element by position; Python starts at index 0.

```
heights = trajectory[:, 0]
```

Control question: before running the example, predict the result type and shape.

4. negative index

Syntax: `values[-1]`

Select from the end; -1 is the last element.

```
last_state = trajectory[-1]
```

Control question: before running the example, predict the result type and shape.

5. slicing

Syntax: `values[a:b]`

Select a subsequence; b is excluded.

```
subset = values[1:4]
```

Control question: before running the example, predict the result type and shape.

6. tuple

Syntax: `state = (h,v,t)`

Ordered container typically used here as a compact state snapshot.

```
state = (h, v, t)
```

Control question: before running the example, predict the result type and shape.

7. mutability

Syntax: `list mutable / tuple container immutable`

Lists can change in place; tuple item references cannot be reassigned.

```
values = [1, 2]
values[0] = 99
```

Control question: before running the example, predict the result type and shape.

8. aliasing

Syntax: `a = b`

Two names may refer to the same mutable object.

```
a = [1, 2]
b = a
b.append(3)
```

Control question: before running the example, predict the result type and shape.

9. generator expression

Syntax: `max(s[0] for s in trajectory)`

Generate values lazily for functions such as max().

```
max_h = max(state[0] for state in trajectory)
```

Control question: before running the example, predict the result type and shape.

10. lambda

Syntax: `lambda s: s[0]`

Small anonymous function; useful as key= in some operations.

```
highest = max(trajectory, key=lambda state: state[0])
```

Control question: before running the example, predict the result type and shape.

Common mistakes

- Confusing a function name with a function call using parentheses.
- Not checking types, indices or shapes before an operation.
- Using a tool because it works without being able to explain what it receives and returns.

Mental cheat sheet

```
list: values = []
append(): values.append(x)
indexing: values[i]
negative index: values[-1]
slicing: values[a:b]
tuple: state = (h,v,t)
mutability: list mutable / tuple container immutable
aliasing: a = b
generator expression: max(s[0] for s in trajectory)
lambda: lambda s: s[0]
```